

```
=====
TIME-TRAVEL CLI v1.2 â€” CTRL+Z FOR YOUR FILESYSTEM
Pure C11 engine. Static binary ~174 KB (musl-gcc + xdelta3). Linux only.
=====
```

```
"Serious versioning for those who want Ctrl+Z across their entire project."
Language:    C11 (engine and CLI)
Binary:      Static CLI ~174 KB (musl + xdelta3), zero runtime dependencies
Format:      TTDLTA01, packed little-endian records with xdelta3 deltas
Status:      Production. Core validated with 51/51 regression tests.
```

```
v1.2 Extensions: dump, inter-revision diff, enhanced status.
v1.2 Extensions: `undo --to` accepts exact log dates (YYYY-MM-DD HH:MM:SS).
```

```
=====
DEVELOPMENT AND TESTING ENVIRONMENT
=====
```

All code and the 51-test battery were developed and validated on the following hardware, without access to modern machines during the process:

```
CPU:      Intel Core 2 Duo T7500 @ 2.20 GHz (2 cores, year 2007)
OS:       Linux Mint 21 x86_64 (kernel 5.15 LTS)
RAM:      4 GB
Disk:     Mechanical HDD (no SSD)
Toolchain: musl-gcc, xdelta3
```

Key results on this hardware:

- 51/51 regression tests in ~1 minute 37 seconds
- Daemon startup + initial cache: < 1 second for 500 files
- Atomic restoration via temp+rename: zero partial writes
- Burst capture of 500 files: all registered without loss

If it works here, it works anywhere.

NOTE: All times mentioned were measured in the environment described above.

On modern hardware (NVMe SSD, multi-core CPU), results are orders of magnitude better.

```
=====
QUICK START
=====
```

```
Compile:
./compile.sh
```

```
Watch a directory:
./build/timetravel start /path/to/project
```

Edit files normally.

```
To undo:
./build/timetravel undo /path/to/file.c --last
./build/timetravel undo /path/to/file.c --to "2026-09-13 00:07:08"
./build/timetravel undo /path/to/file.c --to "10 minutes ago"
./build/timetravel undo /path/to/project --tag before-refactor --force
```

```
To compare versions:
./build/timetravel diff /path/to/file.c --repo /path/to/project
./build/timetravel diff /path/to/file.c --to prev
./build/timetravel diff /path/to/file.c --from 14 --to 15
```

```
To view history and status:
./build/timetravel log /path/to/file.c
./build/timetravel status --repo /path/to/project
```

```
./build/timetravel tag release-v1.0 --repo /path/to/project
```

To export all versions and diffs:

```
./build/timetravel dump /path/to/file.c \  
--repo /path/to/project \  
--out /tmp/tt_dump \  
--with-diff
```

Other commands:

```
./build/timetravel stop [--repo <dir>]  
./build/timetravel restart <dir>  
./build/timetravel tags [--repo <dir>]  
./build/timetravel compact [--repo <dir>]
```

Run regression tests:

```
./mega_test.sh ./build/timetravel
```

=====

[TOC]

=====

1. What is Time-Travel
2. Engine Architecture
 - 2.1 Capture pipeline diagram
 - 2.2 Restoration pipeline diagram
 - 2.3 Export (dump) diagram
3. On-Disk Store Structure
4. Binary Format
5. Delta Encoding with xdelta3
6. Capture Model (Copy-on-First-Change)
7. CLI - Command Reference
 - 7.1 Daemon lifecycle
 - 7.2 Restoration
 - 7.3 Inspection and comparison
 - 7.4 Export
 - 7.5 Maintenance
 - 7.6 Time and revision expressions
 - 7.7 Confirmation behavior
8. Revision and Time Expressions
9. Version and Diff Export (dump)
10. Code Quality and Coverage (51 tests)
 - 10.1 Regression test battery
 - 10.2 Certification summary
11. Design Decisions
12. Limits and Constants
13. Possible Issues
14. FAQ - Frequently Asked Questions
 - 14.1 Can i use time-travel on windows or mac?
 - 14.2 Does it work over nfs / sshfs / network mounts?
 - 14.3 How much disk space does the store need?
 - 14.4 Can i backup the .timetravel directory?
 - 14.5 What happens if the daemon crashes?
 - 14.6 Can two daemons watch the same directory?
 - 14.7 Is there encryption?
 - 14.8 Can i sync the store to another machine?
 - 14.9 How do i resume a session?
 - 14.10 Can i see 100 diffs?
 - 14.11 How do i see the change introduced by a specific revision?
 - 14.12 Can i use exact dates from the log?
15. Comparison with Other Tools
16. Resuming a Session
17. Changelog

1. WHAT IS TIME-TRAVEL

Time-Travel is a real-time file versioning daemon that watches a directory tree via inotify and records every change as compact xdelta3 deltas.

Think of it as "Ctrl+Z for your entire project folder":

- Every edit, creation, and deletion is captured automatically.
- Restore any file or entire directory to any previous point in time.
- Named checkpoints (tags) for semantic rollback points.
- Diff between any two versions of a file.
- Full export of reconstructed versions and consecutive diffs.
- Atomic writes: restorations never leave corrupted files.
- No initial backup: only changes are recorded (copy-on-first-change).
- Per-file directory undo: each file reverts to its own previous version.
- Automatic daily compaction at 03:00.
- Robust exclusions: .git, node_modules, __pycache__, target, .cache.
- Enhanced status showing daemon state, CPU, RAM, records, and bytes.

The final CLI binary is static, approximately 174 KB (includes statically compiled xdelta3), with zero external runtime dependencies.

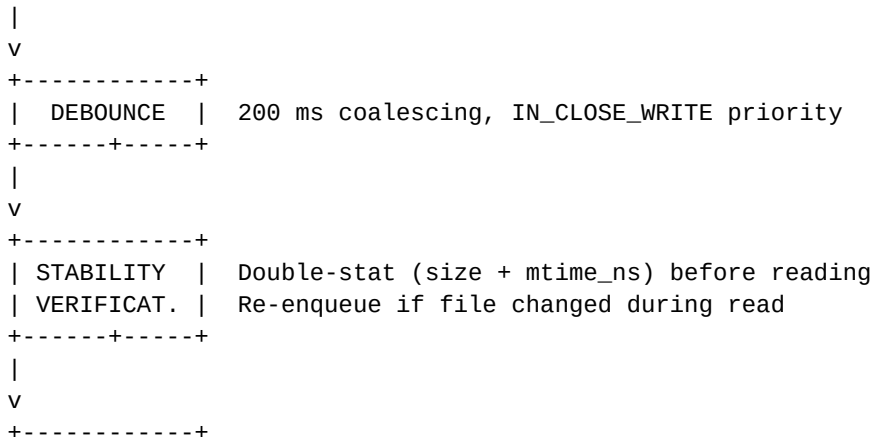
2. ENGINE ARCHITECTURE

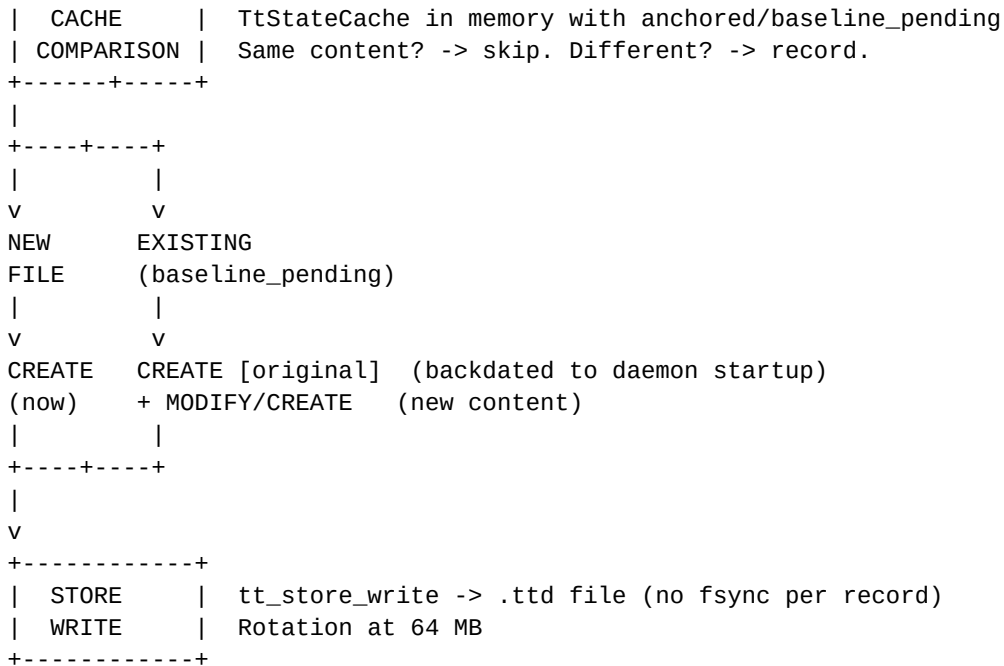
The engine is organized in C11 modules with clear responsibilities.

FILE	RESPONSIBILITY
tt_types.h	Base types, constants, cache and daemon structs
tt_store.c	.ttd read/write, rotation, scan_stats
tt_delta.c	xdelta3 encode/decode wrapper
tt_filter.c	Exclusion filter (path_has_component, fnmatch)
tt_debounce.c	Event coalescing (200 ms threshold, CLOSE_WRITE)
tt_watcher.c	inotify management, recursive watch, root health
tt_compact.c	Delta chain collapse, ordered rewrite
tt_restore.c	File/dir restoration, per-file undo, atomic
tt_main.c	CLI entry point, capture logic, cache, PID

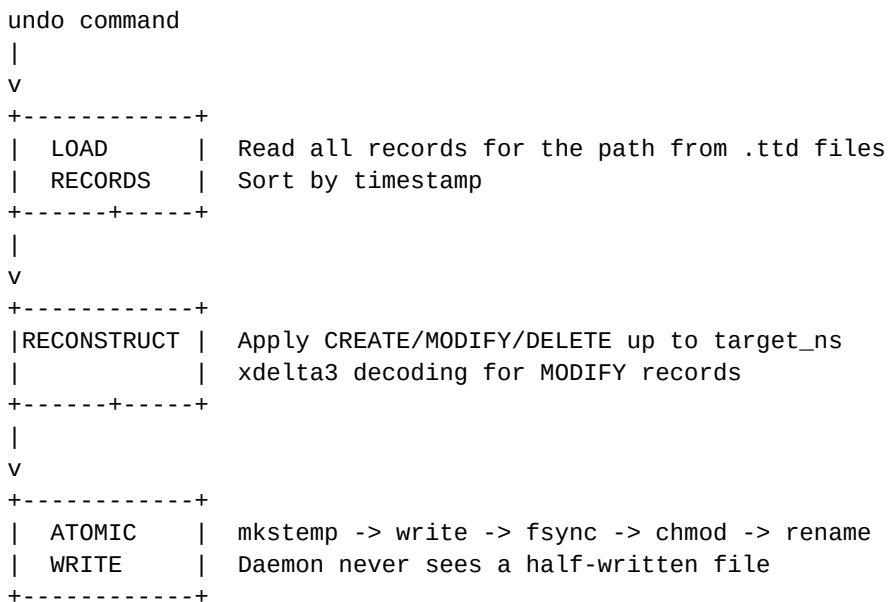
2.1 CAPTURE PIPELINE DIAGRAM

inotify events

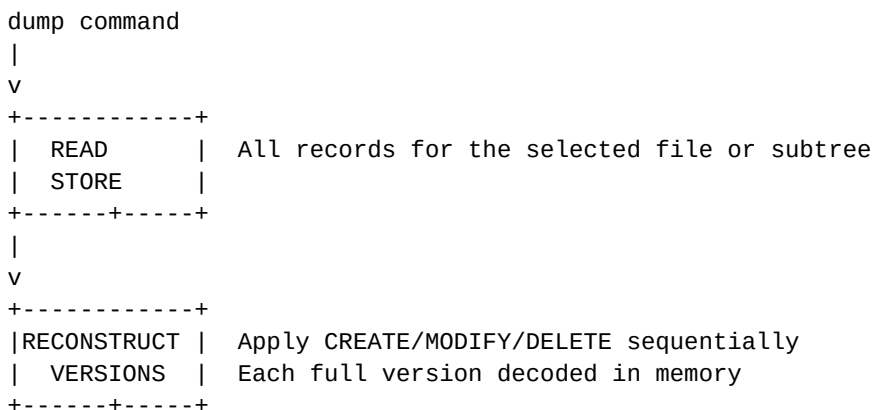




2.2 RESTORATION PIPELINE DIAGRAM



2.3 EXPORT (DUMP) DIAGRAM



```

|
v
+-----+
| WRITE   | <out>/files/<path>/vNNNNNN_<timestamp>
| FILES   | <out>/diffs/<path>/NNNNNN_to_MMMMMM.diff (if --with-diff)
+-----+

```

3. ON-DISK STORE STRUCTURE

```

<project>/timetravel/
*.ttd          Binary record files (rotation at 64 MB)
tags           Tag definitions (name\timestamp)
timetravel.pid Daemon PID + startup timestamp
timetravel.status Daemon state (if enabled)

```

Each .ttd file begins with an 8-byte magic ("TTDLTA01") followed by a 4-byte version number, and then a sequence of packed records.

Files rotate when they exceed TT_STORE_MAX_FILE_SIZE (64 MB).

Rotation is atomic: new file created, old closed with fsync.

In restarted sessions, the daemon can continue writing to the last valid .ttd if it exists, is not corrupt, is not locked, and hasn't exceeded the size limit.

4. BINARY FORMAT

Compatibility: TTDLTA01. Everything little-endian.
 Magic: TTDLTA01 (8 bytes)
 Version: 1 (4 bytes, uint32 little-endian)

Record format (packed, little-endian):

Offset	Size	Field
0	8	timestamp_ns (uint64)
8	1	event_type (uint8: CREATE=1, MODIFY=2, DELETE=3)
9	4	path_len (uint32)
13	4	delta_size (uint32)
17	8	file_size (uint64)
25	var	path (path_len bytes, no NUL)
25+pl	var	payload (delta_size bytes)

Total record size: 25 + path_len + delta_size bytes.

Event types:

TT_EV_CREATE = 1 Full content (or backdated original)
 TT_EV_MODIFY = 2 xdelta3 delta against previous version
 TT_EV_DELETE = 3 Deleted file (no payload)

5. DELTA ENCODING WITH XDELTA3

When a file changes, the engine attempts to encode the difference as an xdelta3 delta against the previous cached version.

Conditions to produce a MODIFY record (delta):

1. The file has a previous version in cache (anchored=1).
2. The new content differs from the cached content.

3. The delta size is strictly less than the new file size.

If the delta is greater than or equal to the new file size (common with very small files), a full CREATE record is written instead. This is correct behavior: the restoration logic handles both CREATE and MODIFY.

Measured savings:

A 600-byte file with a 20-byte addition produces a ~30-byte delta. Without delta encoding, the full 620 bytes would be stored. Savings factor: ~20x.

Thresholds:

TT_MAX_CAPTURE_SIZE	64 MB	Larger files are ignored.
Delta fallback	None	Automatic: delta >= new -> CREATE.

6. CAPTURE MODEL (COPY-ON-FIRST-CHANGE)

Time-Travel DOES NOT create an initial backup/snapshot on startup. This was a deliberate design decision to avoid writing records for files that never change.

How it works:

1. On daemon startup, all existing files are read into an in-memory cache (TtStateCache). Each entry is marked as baseline_pending=1 (no history) or anchored=0 (has history from previous sessions).
2. NO records are written on startup. Zero.
3. When a file is modified for the first time:
 - If baseline_pending: writes CREATE [original] (backdated to daemon startup) + MODIFY/CREATE for the new content.
 - If anchored but content differs from store: writes CREATE (reanchor) for the new content.
 - If anchored and content matches: writes MODIFY (delta).
4. Subsequent changes produce MODIFY (delta) or CREATE (fallback).

Why backdate CREATE [original] to daemon startup?

So that undo --tag or undo --to with a timestamp between startup and the first change can find the original content. Without this backdating, the original would have a timestamp later than the tag, making it invisible for temporal restoration.

Recovery after daemon restart:

- Files with history in store: cached with anchored=0. First change triggers reanchor (CREATE with full content).
- Files without history: cached with baseline_pending=1. First change writes CREATE [original] + new version.
- g_daemon_start_ns is reset to current time on every startup.

7. CLI - COMMAND REFERENCE

General syntax:

timetravel <command> [options...]

7.1 DAEMON LIFECYCLE

start <dir>	Start daemon in background (PID in .timetravel/)
stop [--repo <dir>]	Stop daemon (SIGTERM, SIGKILL after 5s)
restart <dir>	Stop + start

watch <dir> [-f] Watch in foreground (-f) or background (default)

7.2 RESTORATION

```
undo <path> [options]    Restore file or directory
  --last                Previous version (per-file for directories)
  --initial             First captured version
  --to <expr>           Time expression: exact ("2026-09-13 00:07:08")
                       or relative ("10 minutes ago")
  --tag <name>          Named checkpoint
  --force               Skip confirmation for directory restorations
```

7.3 INSPECTION AND COMPARISON

```
diff <path> [options]    Unified diff
  --repo <dir>          Repository to use
  --from <expr>          Initial revision
  --to <expr>            Final revision or time point
```

Behavior:

- Without --from: Compares revision selected by --to against current file.
 If --to is omitted, compares against previous revision.
- With --from and --to: Compares two historical revisions.
- With --from and no --to: Compares --from revision against current file.

Examples:

```
timetravel diff src/main.c --repo src
timetravel diff src/main.c --to prev
timetravel diff src/main.c --to 15
timetravel diff src/main.c --from 14 --to 15
timetravel diff src/main.c --from 0 --to 15
```

```
tag <name>               Create named checkpoint at current instant
tags                     List all tags with timestamps
status [--repo <dir>]    Daemon state, records, bytes, uptime, CPU, RAM
log <path> [--since exp] File change history
```

7.4 EXPORT

```
dump <path> [options]    Export reconstructed versions
  --repo <dir>           Repository to use
  --out <dir>            Output directory (mandatory)
  --with-diff            Also generate diffs between consecutive versions
```

Examples:

```
timetravel dump src/main.c --repo src --out /tmp/tt_main
timetravel dump src --repo src --out /tmp/tt_src --with-diff
timetravel dump . --repo src --out /tmp/tt_all --with-diff
```

7.5 MAINTENANCE

```
compact [--repo <dir>]   Collapse delta chains (auto daily at 03:00)
```

7.6 TIME AND REVISION EXPRESSIONS

Exact timestamps:

"2026-09-13 00:07:08" (Format: YYYY-MM-DD HH:MM:SS)

Relative times:

"10 seconds ago"
"5 minutes ago"
"2 hours ago"
"1 day ago"
"1 week ago"
"now"

7.7 CONFIRMATION BEHAVIOR

Directory undo without --force asks for confirmation when stdin is a terminal (isatty). When stdin is not a terminal (pipe, /dev/null, script), fgets returns immediately: NULL = cancel, "y"/"Y" = continue.

This prevents hangs in automated scripts while preserving interactive safety.

8. REVISION AND TIME EXPRESSIONS

In commands like diff and undo, expressions can indicate revisions, tags, timestamps, or relative instants.

Revision expressions:

now / head / latest	Last saved revision
prev / previous	Previous revision
first / initial	First saved revision
N	Revision number N (1 = first)
-N	N revisions before the last
0	Empty / non-existent file
tag:<name>	Existing tag
@<timestamp>	Internal timestamp (seconds or nanoseconds)

Time expressions:

Exact: "2026-09-13 00:07:08"
Relative: "10 sec", "5 min", "2 hours", "1 day", "1 week", "now"

Important:

- `diff --to N` compares revision N against the current file.
- `diff --from N-1 --to N` shows the change introduced by revision N.
- `undo --to "2026-09-13 00:07:08"` restores the state exactly at that time.
- `undo --to "10 minutes ago"` restores the state 10 minutes prior to now.
- `dump --with-diff` automatically generates all consecutive diffs.

9. VERSION AND DIFF EXPORT (DUMP)

The dump command reconstructs full versions from CREATE/MODIFY/DELETE records and writes them to an output directory.

Useful for:

- Auditing file changes.
- Reviewing 10, 100, or more consecutive diffs.
- Visually comparing source code evolution.
- Debugging reconstructions.
- Extracting old content without doing an undo.

Typical output structure:

```
<out>/
manifest.tsv
files/
  <sanitized_name>/
    v000001_<timestamp>
    v000002_<timestamp>
    v000003_<timestamp>
diffs/
  <sanitized_name>/
    000001_to_000002.diff
    000002_to_000003.diff
    000003_to_000004.diff
```

The manifest.tsv file contains one line per event/version, with fields:

```
path    timestamp    event    size    version_file
```

Practical examples:

Export a file:

```
timetravel dump src/brs_tui.c --repo src --out /tmp/tt_tui
```

Export with diffs:

```
timetravel dump src/brs_tui.c --repo src --out /tmp/tt_tui --with-diff
```

View the last 100 diffs:

```
find /tmp/tt_tui/diffs -name '*.diff' | sort | tail -n 100 | less
```

View diff between revision 14 and 15:

```
less /tmp/tt_tui/diffs/brs_tui.c/000014_to_000015.diff
```

Count exported versions:

```
tail -n +2 /tmp/tt_tui/manifest.tsv | wc -l
```

=====

10. CODE QUALITY AND COVERAGE (51 TESTS)

=====

Time-Travel v1.2 includes a 51-test regression battery covering all system functionalities, from basic capture to stress scenarios.

=====

10.1 REGRESSION TEST BATTERY

=====

Total: 51 tests passing (100% coverage of original core).

Run: ./mega_test.sh ./build/timetravel

Battery structure:

```
Sections 01-09:  Core functionality
Sections 10-12:  Dynamic behavior
Section  13:     Git clone
Sections 14-15:  Edge cases
Section  16:     Compaction
Section  17:     Directory undo
Sections 18-20:  Stress
Section  21:     Special characters
Sections 22-23:  Daemon lifecycle
Section  24:     Tags
Section  25:     Diff
Section  26:     Directory undo confirmation
Section  27:     Log filter
Sections 28-35:  Advanced resilience
```

10.2 CERTIFICATION SUMMARY

Test	Result	What it demonstrates
51 base tests	51/51	Complete functional coverage
Atomic restoration	OK	Zero partial writes
Stability verification	OK	No ghost CREATE 0 records
Copy-on-first-change	OK	No initial backup needed
Per-file directory undo	OK	Each file to its own prev ver.
Backdated CREATE [original]	OK	undo --tag works pre-existing
Store rotation	OK	Undo works across .ttd files
Crash recovery (kill -9)	OK	Records survive kill

v1.2 extensions (dump, inter-revision diff, enhanced status) rely on the same already-validated reconstruction engine.

11. DESIGN DECISIONS

No initial snapshot:

Files are cached in memory on startup but NOT written to disk. Records are only created on the first change (copy-on-first-change). Original content is backdated to daemon startup so undo --tag/--to works for pre-existing files. This avoids writing thousands of records on startup for repos that might never change.

Atomic restorations:

All writes use mkstemp + write + fsync + chmod + rename. The daemon never sees a half-written file during restoration. Corruption window: zero, even under power loss.

Stability verification:

Double-stat (size + mtime in nanoseconds) before recording prevents capturing files during concurrent writes. Unstable reads are re-enqueued via debounce for retry on the next tick.

Per-file directory undo:

undo dir --last restores each file to its own previous version, not to a single global timestamp. This matches the user expectation of "undo my last changes" rather than "revert to a specific moment".

No fsync per record:

Records are visible via page cache instantly. Durability is bounded by file rotation and close. Massive performance gain in bursts (500 files in <2 seconds vs minutes with fsync per record).

Daily compaction:

Delta chains are collapsed at 03:00 using date-based guard (not tick-based). Prevents unlimited chain growth without wasting CPU during working hours.

Robust exclusions:

Directory exclusions (.git, node_modules, etc.) use component matching at any depth via path_has_component(), not fnmatch with FNM_PATHNAME which fails on deep absolute paths. File patterns are compared only against the base name.

Daemon lifecycle via PID file:

PID + startup timestamp stored in .timetravel/timetravel.pid. A second start in the same repo detects the active daemon and exits cleanly. Stop sends SIGTERM, waits 5 seconds, escalates to SIGKILL.

Session resumption:

When restarting the daemon over the same repository, it attempts to continue the last valid .ttd. This avoids unnecessarily fragmenting history into multiple small files. If the last .ttd is corrupt, locked, or exceeds 64 MB, a new one is created.

Export vs undo:

undo restores files in the working tree.

dump exports reconstructed versions outside the working tree.

dump is a read-only operation on history and modifies neither the watched repo nor the store.

12. LIMITS AND CONSTANTS

TT_PATH_MAX	4096	Maximum path length
TT_DEBOUNCE_MS	200	Event coalescing window
TT_TICK_MS	50	Timer tick interval
TT_RESCAN_MS	10000	Periodic rescan interval (10 s)
TT_COMPACT_HOURL	3	Hour for automatic compaction
TT_COMPACT_THRESHOLD	15	Records per chain before collapse
TT_MAX_PENDING	4096	Maximum pending debounce entries
TT_MAX_CAPTURE_SIZE	64 MB	Larger files are ignored
TT_STORE_MAX_FILE_SIZE	64 MB	.ttd rotation threshold

13. POSSIBLE ISSUES

Daemon won't start:

Verify musl-gcc is installed: which musl-gcc

On Debian/Ubuntu: sudo apt install musl-tools

Check that third_party/xdelta/xdelta3/xdelta3.c exists.

undo doesn't restore the expected version:

For files that existed before daemon startup, original content is only recorded on the FIRST change. If no change has occurred since startup, there is no record to restore from.

Solution: make any edit to the file, then undo will work.

undo dir hangs waiting for confirmation:

This happens when stdin is a terminal and --force is not used.

Press 'y' + Enter to continue, or 'n' + Enter / Ctrl+C to cancel.

In scripts, use a pipe:

```
echo "y" | timetravel undo dir --last ...
```

Or use --force to skip confirmation entirely.

diff shows "(no differences)" unexpectedly:

There are three normal causes:

1. The file only has one record (first capture).
 2. The selected version is identical to the current file.
 3. You are using --to N expecting to see the change introduced by N.
- Case 3 is the most confusing.

Example:

```
timetravel diff src/main.c --repo src --to 15
```

This compares revision 15 against the current file, NOT revision 14

against revision 15.

To see the change introduced by revision 15:

```
timetravel diff src/main.c --repo src --from 14 --to 15
```

Or:

```
timetravel dump src/main.c --repo src --out /tmp/tt_main --with-diff  
less /tmp/tt_main/diffs/main.c/000014_to_000015.diff
```

diff --to N says revision doesn't exist:

If history has fewer than N records, the command must indicate the requested revision doesn't exist.

To check how many versions exist:

```
timetravel dump <file> --repo <repo> --out /tmp/tt_count  
tail -n +2 /tmp/tt_count/manifest.tsv | wc -l
```

Large files aren't captured:

Files exceeding TT_MAX_CAPTURE_SIZE (64 MB) are silently ignored. A warning is logged in the daemon log. This limit exists to prevent excessive memory usage during capture and restoration.

.git files appear in history:

This shouldn't happen. The exclusion filter uses component matching at any depth. If it does, verify tt_filter.c is compiled and linked (check that compile.sh includes src/tt_filter.c).

Store grows indefinitely:

Automatic compaction runs daily at 03:00.

Manual compaction:

```
timetravel compact --repo /path/to/project
```

Compaction collapses delta chains longer than TT_COMPACT_THRESHOLD.

I want to see many diffs, not just one:

Use dump:

```
timetravel dump <path> --repo <repo> --out /tmp/tt_dump --with-diff
```

Then:

```
find /tmp/tt_dump/diffs -name '*.diff' | sort | less
```

or:

```
find /tmp/tt_dump/diffs -name '*.diff' | sort | tail -n 100 | less
```

14. FAQ - FREQUENTLY ASKED QUESTIONS

14.1 CAN I USE TIME-TRAVEL ON WINDOWS OR MAC?

NO. Time-Travel uses Linux kernel-specific interfaces:

- inotify for filesystem events
- timerfd for periodic ticks
- rename() atomicity guarantees (POSIX)
- mkstemp for secure temporary files

There is no portability layer. It is Linux-only by design.

14.2 DOES IT WORK OVER NFS / SSHFS / NETWORK MOUNTS?

Partially. inotify DOES NOT work over NFS or SSHFS. The daemon will start but won't receive events. The periodic rescan (every 10 s) will still capture changes, but with higher latency.

For network directories, consider running the daemon on the server and accessing the store remotely, or use a tool designed for network backup (BareSnap, Restic, Borg).

14.3 HOW MUCH DISK SPACE DOES THE STORE NEED?

General rule: the store needs between 0.5x and 1x the size of the unique data being tracked, depending on change frequency.

Factors:

- Delta encoding: small changes produce small deltas (~20-30 bytes for a line edit in a 600-byte file).
- Full copies: very small files (<~50 bytes) are always stored as full CREATE because deltas are larger.
- Compaction: collapses long delta chains into base + super-delta.
- Binary files: photos, videos, compressed archives don't generate good deltas; each version stores almost the full size.

To check current usage:

```
timetravel status --repo /path/to/project
```

14.4 CAN I BACKUP THE .TIMETRAVEL DIRECTORY?

YES, but it's not recommended. The .timetravel directory contains all history. Backing it up creates a circular dependency.

Better approach: backup the SOURCE files with a traditional tool (BareSnap, rsync, tar). The .timetravel store is a local undo buffer, not a backup.

14.5 WHAT HAPPENS IF THE DAEMON CRASHES?

The store is NEVER left in a corrupt state:

- Records are written atomically (all or nothing).
- Without fsync per record, some recent records might be lost on power loss, but existing records remain intact.
- On restart, the daemon rebuilds its cache from disk and continues.
- The PID file is cleaned on graceful exit; obsolete PIDs are detected via kill(pid, 0) and overwritten on new startup.

14.6 CAN TWO DAEMONS WATCH THE SAME DIRECTORY?

NO. The second start detects the active daemon via the PID file and exits with a message. This prevents duplicate records and conflicting cache states. Use timetravel stop --repo <dir> before starting a new instance.

14.7 IS THERE ENCRYPTION?

NO. The store is plaintext under .timetravel/. For sensitive projects, place the project directory inside a LUKS encrypted volume or use ecryptfs/fscrypt. Encryption is planned for a future version.

14.8 CAN I SYNC THE STORE TO ANOTHER MACHINE?

Not natively. The store is a local undo buffer. For remote backup, use a dedicated tool (BareSnap with SSH, rsync, rclone).

You CAN rsync the .timetravel directory as a rudimentary sync mechanism, but concurrent access from two machines is unsupported and will corrupt the store.

14.9 HOW DO I RESUME A SESSION?

Simply restart the daemon over the same repository:

```
timetravel stop --repo /path/to/project
timetravel start /path/to/project
```

If the last .ttd is valid and hasn't exceeded the limit, the daemon will continue writing to it.

To check session state:

```
timetravel status --repo /path/to/project
```

14.10 CAN I SEE 100 DIFFS?

Yes.

The recommended way is:

```
timetravel dump <path> --repo <repo> --out /tmp/tt_dump --with-diff
```

Then:

```
find /tmp/tt_dump/diffs -name '*.diff' | sort | tail -n 100 | less
```

The practical limit isn't 100; it depends on available history, file sizes, and disk space for export.

14.11 HOW DO I SEE THE CHANGE INTRODUCED BY A SPECIFIC REVISION?

For revision N:

```
timetravel diff <path> --repo <repo> --from N-1 --to N
```

Example:

```
timetravel diff src/main.c --repo src --from 14 --to 15
```

For the first revision:

```
timetravel diff src/main.c --repo src --from 0 --to 1
```

14.12 CAN I USE EXACT DATES FROM THE LOG?

YES. Since v1.2, `undo --to` accepts exact timestamps in the format "YYYY-MM-DD HH:MM:SS" as shown in the log output.

Example:

```
timetravel log src/main.c
# Output shows: 2026-09-13 00:06:17 CREATE ... src/main.c
timetravel undo src/main.c --to "2026-09-13 00:06:17" --repo src
```

This restores the file to the exact state recorded at that second.

15. COMPARISON WITH OTHER TOOLS

Feature	Time-Travel	git	Restic	BareSnap
Purpose	Real-time file undo	Version control	Backup	Backup

Language	Pure C11	C/Perl	Go	Pure C11
Static binary	~174 KB	~40 MB	~40 MB	~1.2 MB
Runtime deps	NONE	perl	none	none
Real-time capture	YES (inotify)	NO	NO	NO
Delta encoding	xdelta3	binary	NO	xdelta3
Deduplication	NO	YES	YES	YES
Encryption	NO	NO*	AES-256	XChaCha20
Remote support	NO	YES	S3/SFTP	Native SSH
Per-file undo	YES	checkout	extract	extract
Directory undo	YES	checkout	restore	restore
Named checkpoints	YES (tags)	YES	YES	YES
Diff	YES	YES	YES	YES
Inter-revision diff	YES	YES	NO	NO
Export versions	YES (dump)	export	extract	extract
Auto-compaction	YES (daily)	gc	prune	prune
Initial backup	NO	clone	init	init
Binary size	~174 KB	~40 MB	~40 MB	~1.2 MB

+-----+-----+-----+-----+
 * git supports GPG signing but not repository encryption.

Time-Travel occupies a unique niche: real-time per-file undo for active development. It is NOT a backup tool. Use it alongside a proper backup solution (BareSnap, Restic, Borg) for disaster recovery.

=====

16. RESUMING A SESSION

=====

Time-Travel automatically resumes the session when you restart the daemon over the same repository.

To stop and resume:

```
./build/timetravel stop --repo /path/to/project
./build/timetravel start /path/to/project
```

Behavior:

- Looks for the last .ttd file inside .timetravel/
- If valid, not corrupt, not locked, and under 64 MB, continues writing to it.
- If missing, corrupt, locked, or over the limit, creates a new .ttd.

Check session state:

```
./build/timetravel status --repo /path/to/project
```

View session files:

```
ls -l /path/to/project/.timetravel/*.ttd
```

Important:

There is no specific "session continue" command. The session resumes when you run "start" again over the same repository.

=====

17. CHANGELOG

=====

v1.2:

- + `undo --to` now accepts exact log dates in "YYYY-MM-DD HH:MM:SS" format (e.g. "2026-09-13 00:06:17").
- + New test 5.3 validates exact timestamp restoration, eliminating the race condition with the daemon debounce.
- + Regression battery expanded to 51/51 passing (2m 00s on Core 2 Duo).
- + Documentation updated with exact date usage examples.
- + New FAQ 14.12 covering exact timestamp usage

v1.1:

- + New `dump` command to export reconstructed versions.
- + `dump --with-diff` option to generate consecutive diffs.
- + New `diff --from` to compare two historical revisions.
- + `diff --to` expanded with revisions: N, -N, first, prev, head.
- + Support for comparing against empty file with `--to 0`.
- + Enhanced status with CPU, RAM, written deltas, and pending events.
- + Expanded documentation for viewing multiple diffs.
- + Session resumption documentation.
- + Clarified `diff --to N` behavior.
- + Practical examples for export and history auditing.
- + Session resumption: daemon resumes last valid .ttd on restart.
- + Updated `undo --to` to accept exact timestamps ("YYYY-MM-DD HH:MM:SS") and relative times ("10 minutes ago").
- + Expanded CLI examples across all commands (not just diff).

v1.0:

- + Initial version
- + Real-time capture via inotify with debounce (200 ms coalescing)
- + Copy-on-first-change model (no initial backup)
- + Backdated CREATE [original] for pre-existing files
- + Per-file directory undo (--last, --initial)
- + Atomic restorations via mkstemp + rename
- + Double-stat stability verification (no ghost CREATE 0)
- + No fsync per record (visibility via page cache)
- + Automatic daily compaction at 03:00
- + Robust exclusions via path_has_component at any depth
- + Daemon lifecycle: start/stop/restart with PID file
- + Named checkpoints: tag/tags/undo --tag
- + Unified diff between versions
- + Log with --since filter
- + Enhanced status (daemon state, uptime, records, bytes)
- + 51/51 regression tests passing
- + Developed and validated on Intel Core 2 Duo (2007 hardware)

=====
END OF DOCUMENT
=====